

Лекция 1. Сущность объектно-ориентированного подхода в программировании. Язык C++. Ввод и вывод в C++. Структуры, перечислимые типы

Сущность объектно-ориентированного подхода в программировании

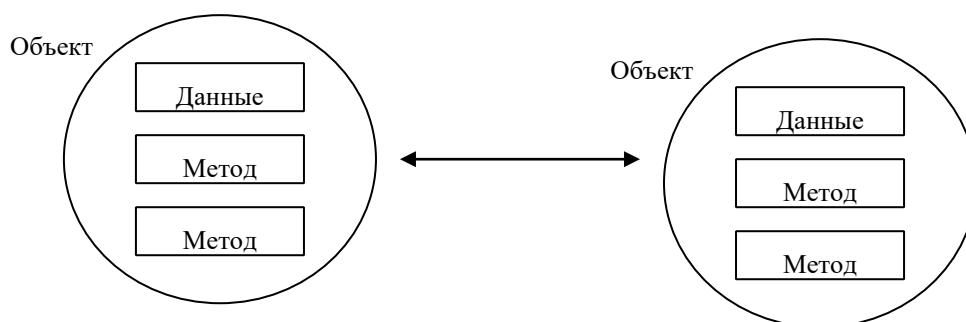
К настоящему времени получили развитие три парадигмы программирования: процедурная, объектно-ориентированная и функциональная. *Процедурное программирование* было исторически первым подходом. При таком построении программ они представляют собой набор данных и процедур, вызывающих друг друга и обрабатывающих данные.

С течением времени проявилась ограниченность процедурного программирования. Задачи, для которых изначально создавались процедурные языки программирования, характеризовались небольшим объемом команд и не требовали дополнительной внутренней организации. Когда же размер программы велик, список команд становится слишком громоздким, и код необходимо разделять на более мелкие логические части. Деление программы на функции и модули является основой процедурного программирования, как способа организации программ. Однако существенно упростить большие программы процедурный подход к программированию позволяет не всегда.

Объектно-ориентированный подход к программированию заключается в моделировании объектов реального мира, интерпретируемых как совокупность свойств и поведения. Примерами свойств для людей могут являться цвет глаз или место работы; для машин – мощность двигателя и количество дверей. В этом смысле свойства объектов равносильны данным в программах, они имеют определенное значение, например, красный для цвета глаз или 4 для количества дверей автомобиля. Поведение – это некоторое действие объекта в ответ на внешнее воздействие. Поведение сходно с функциями – функция вызывается, чтобы совершить какое-либо действие. В процедурных языках программирования данные и функции принципиально разделены, что затрудняет моделирование ими объектов реального мира. Основопологающей же идеей объектно-ориентированного подхода является объединение данных и функций, оперирующих этими данными, в одно целое, которое и называется *объектом*.

Из всех объектно-ориентированных языков C++ долгое время (примерно с начала 1990-х по конец 2000-х годов) являлся наиболее широко употребительным. Другой распространённый сейчас язык Java лишен таких составляющих как указатели, шаблоны и множественное наследование. Наиболее популярными в профессиональной среде средами разработки для C++ являются продукты, разработанные компаниями Microsoft.

Типичная программа на C++ состоит из совокупности объектов, взаимодействующих между собой посредством вызова функций (методов) друг друга. Структура объектно-ориентированной программы на C++ приведена на рисунке.



Основными понятиями объектно-ориентированных языков являются: *объект, класс, наследование, полиморфизм, перегрузка, инкапсуляция*.

Классом называется тип данных, включающий в себя элементы-данные и элементы-функции (методы).

Объектом называется переменная, созданная как представитель класса, то есть имеющая соответствующий классу тип.

Инкапсуляция – это способ ограничения доступа к данным объекта. Если данные объекта инкапсулированы, то доступ к ним имеют лишь функции – методы класса.

Наследование – это способ создания дочерних или производных классов на основе родительских или базовых классов. Базовый класс содержит элементы, общие для производных классов. При простом наследовании дочерний класс создается из единственного родительского путем добавления к нему новых методов и данных. При сложном наследовании у дочернего класса может быть два родителя и у него, таким

образом, будут присутствовать методы и функции обоих родителей. Идея простого наследования схематично показана на рисунке.

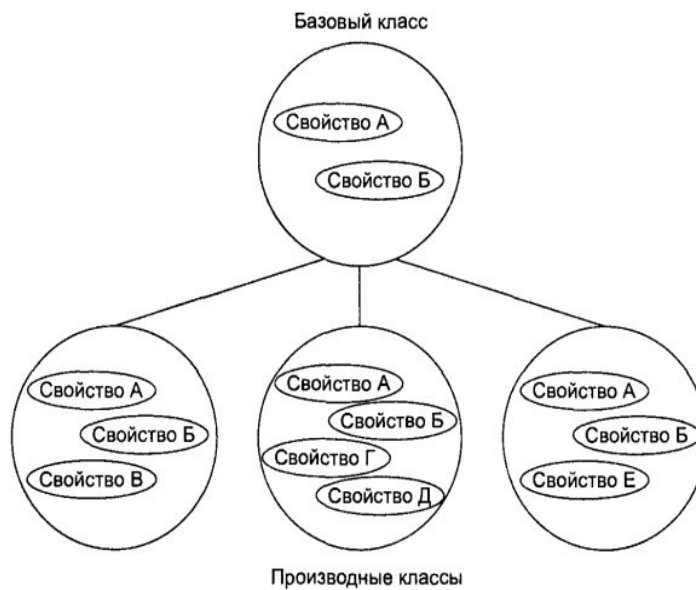
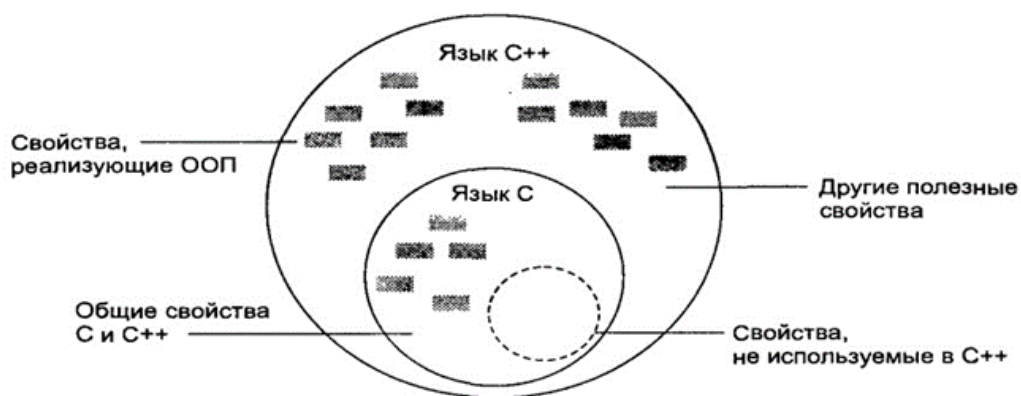


Рис. 1.6. Наследование

Полиморфизм называется такой способ использования функций, когда они выполняют различные действия, в зависимости от того, с какими типами данных они работают. Проявлением полиморфизма являются методы класса, выполняющие различные действия над различными типами данных – то есть перегруженные методы. Ещё одно проявление полиморфизма – методы производного класса, работающие иначе, чем аналогичные методы родительского класса, то есть переопределённые методы.

Язык C++. Ввод и вывод в C++

C++ является расширением языка C. Наиболее значительные нововведения, присутствующие в C++, касаются классов, объектов и ООП. Есть нововведения, касающиеся ввода/вывода. На рисунке приведена иллюстрация отношения между языками C и C++.



Идентификатор *cout* является объектом C++, предназначенным для работы со стандартным потоком вывода. Поток ввода/вывода – это некоторая абстракция, отражающая перемещение данных от источника к приёмнику. Стандартный поток вывода обычно направлен на экран, хотя допускается возможность его перенаправления на другие устройства вывода.

Операция сдвига << перегружена для объекта *cout* как операция вставки. Она копирует содержимое переменной, стоящей в правой её части, в объект, содержащийся в левой её части. Например, команда

```
std::cout << "my string";
```

направляет строку "my string" в переменную *cout*, которая выводит эту строку на экран.

Аналогично *cout* используется идентификатор *cin*, являющийся объектом для работы с потоком ввода. Операция >> перегружена для объекта *cin* как операция извлечения. Она извлекает данные из потокового объекта, стоящего в левой части, и присваивает эти данные переменной, стоящей в правой части. Например, команда

```
std::cin >> temp;
```

заставляет программу ожидать ввода пользователем данных; введённое значение присваивается переменной *temp*.

Идентификаторы *cout* и *cin* являются, таким образом, инструментами ввода-вывода данных, характерных для языка C++. Для доступа к ним необходимо сослаться на заголовочный файл

```
#include <iostream>
```

Полезно также пользоваться директивой, определяющей пространство имен

```
using namespace std;
```

В этом случае становится возможным упрощенное обращение к потокам ввода-вывода, без операции разрешения области действия:

```
cout << "my string";
```

```
cin >> temp;
```

Структуры

В C++ одной из конструкций, реализующих объединение разнородных данных, является структура. Эти данные могут иметь различные типы. Переменные, входящие в состав структуры, называются полями структуры. Структуры в C++ имеют предназначение, сходное с элементом записи некоторых других языков программирования. Пример определения структуры в языке C++:

```
struct Part  
{  
    int modelnumber;  
    int partnumber;  
    float cost;  
};
```

Имея в программе приведенное здесь описание, мы, фактически, определяем новый тип данных *Part*. После определения можно создавать переменные этого типа. Например:

Part p1, p2;

Когда структурная переменная определена, доступ к её полям возможен с помощью операции «точка». Например

```
p1.modelnumber = 12;
```

```
p1.partnumber = 135;
```

```
p1.cost = 52.13;
```

Можно присваивать значение одной структурной переменной другой:

```
p2 = p1;
```

В том случае, если в программе идет работа не с самой структурной переменной, а с указателем на неё, для доступа к полям структуры бывает удобно применять операцию «стрелка»:

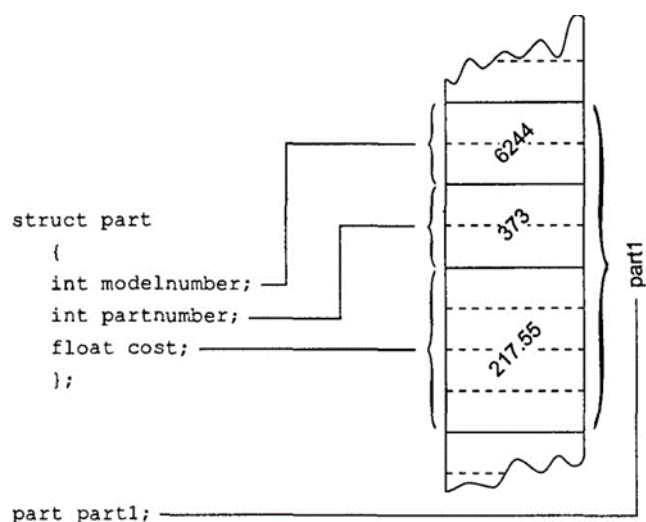
```
Part *z1; // Объявляем указатель на структуру
```

```
z1 = new Part; //выделяем память под размещение структуры
```

```
(*z1).cost = 52.13; // можно обратиться так
```

```
z1->cost = 52.13; // а можно и так
```

При расположении в памяти под структуру отводится объем, необходимый для размещения всех её полей. Поля структуры при этом располагаются одно за другим (см. схему на рисунке).



Структуры допускают вложенность, то есть использование структурной переменной в качестве поля какой-либо другой структуры.

Старые стандарты языка C/C++ допускали и другой синтаксис определения структуры, с помощью оператора определения типов *typedef* (он и был исторически первым). Однако сейчас в основном используется синтаксис, показанный здесь.

Перечисления

Кроме структур, существует ещё один способ создания пользовательских типов данных, называемый перечислением. Использование перечислений является способом упростить процесс создания программ за счет использования мнемонически понятных синонимов для данных.

Перечислениями пользуются в тех случаях, когда переменные создаваемого типа могут принимать заранее известное конечное (и, как правило, небольшое) множество значений. В приведенном ниже примере перечисление используется для создания типа данных, хранящего дни недели.

```
enum DaysOfWeek { SUN, MON, TUE, WED, THU, FRI, SAT };
```

Объявление типа начинается со слова `enum` и содержит перечисление всех возможных значений переменных создаваемого типа. Эти значения называются константами перечисляемого типа. Переменные типа *DaysOfWeek* создаются обычным образом:

```
DaysOfWeek day1, day2;
```

Теперь этим переменным можно присваивать любые определенные для них значения.

```
day1 = SUN;
```

```
day2 = WED;
```

Внутренне перечисляемые типы данных представляются и обрабатываются как целые числа. Это дает возможность применения к переменным перечисляемого типа арифметических операций и операций отношения. Фактически первое значение в списке рассматривается как число 0, второе – как число 1 и т.д. Для того, чтобы изменить значение, с которого начинается нумерация, можно с помощью операции присваивания задать это значение первой из перечисляемых констант.

```
enum DaysOfWeek { SUN = 10, MON, TUE, WED, THU, FRI, SAT };
```

В этом случае следующим по списку за *SUN* константам будут соответствовать числа 11, 12 и т.д. соответственно. Фактически можно изменить величину целого числа, соответствующего любому элементу списка, применив к нему операцию присваивания.

Важным недостатком перечислений является то, что они не распознаются средствами ввода/вывода C++. При работе с ними операциями ввода/вывода они интерпретируются как соответствующие целые числа.

В новых стандартах языка C++ введена (следуя за другими языками программирования) расширенная интерпретация перечислимых типов. Теперь элементами перечисления могут быть не только числа, а данные любого типа. В этом примере константы *RED*, *GREEN* имеют булевский тип:

```
enum class Alert : bool { RED, GREEN };
```

Некоторые версии компиляторов при объявлении перечислений определяют соответствующее им пространство имён. В этом случае следует обращаться к константам перечислений, используя операцию разрешения области действия (продолжим предыдущий пример):

```
bool isRed = Alert::RED;
```

Вопросы к лекции 1

1. Как называется защита данных класса от несанкционированного доступа другими функциями?
2. Как называется возможность выполнения оператором или функцией различных действий в зависимости от типа операндов?
3. Операция, выполняющая заданные действия не только над встроенным, но и над пользовательским типом данных называется:
 - а) полиморфической;
 - б) инкапсулированной;
 - в) классифицированной;
 - г) перегруженной.
4. Структура объединяет:
 - а) данные одного типа;
 - б) логически связанные данные;
 - в) целые именованные значения;
 - г) переменные.

5. Какой заголовочный файл нужно включить в исходный текст, чтобы использовать объекты *cin* и *cout*.
6. Напишите оператор, который получает с клавиатуры числовое значение и присваивает его переменной *temp*.
7. Опишите структуру, содержащую три переменные типа *int* с названиями *hrs*, *mins* и *secs*. Назовите структуру именем *Time*.
8. Истинно ли следующее утверждение: при определении структуры выделяется память под переменную?
9. Напишите оператор, присваивающий полю *hrs* структурной переменной *time2* значение, равное 11.
10. Сколько байтов памяти займут три структурные переменные типа *Time*, если структура *Time* содержит три поля типа *int*.
11. Истинно ли следующее утверждение: вы можете присвоить значение одной структурной переменной другой структурной переменной того же типа?
12. Перечисление объединяет
 - а) данные различных типов;
 - б) логически связанные переменные;
 - в) именованные целые числа;
 - г) константные значения.
13. Напишите оператор, описывающий перечисление с именем *Players* и набором значений *B1*, *B2*, *SS*, *B3*, *RF*, *CF*, *LF*, *P*, *C*.
14. Считая, что перечисление *Players* задано так. Как в предыдущем вопросе, определите переменные *joe* и *tom* типа *Players* и присвойте им значения *LF* и *P* соответственно.

15. Первые три константы перечисляемого типа обычно представляются числами __, __, и __.

16. Напишите оператор, в котором объявляется перечисляемый тип *Speeds* с константами *OBSOLETE*, *SINGLE* и *ALBUM*. Присвойте этим константам целые значения 78, 45, 33.

17. Объясните, почему объявление в языке C++ *enum IsWord {NO, YES};* более удачно, чем *enum IsWord {YES, NO};*